Big Data Analytics (SOC4170): HW1 Solutions, Fall 2026

Every answer below was checked with a small Python script (Apriori, PageRank, Jaccard, entropy, and the confusion-matrix metrics). Each part has a **Source** line with the slide pages the method comes from (W3 p.65 = week 3 slides, page 65; plain text, for reference) and a **Notes** link that opens the md notes section explaining it. For the fastest learning, read that notes section, then the slide, then the solution.

Conventions are the course slides' conventions:

- **Apriori:** self-join, then prune, then scan; "frequent" means support ≥ min support (weeks 3–4).
- **PageRank:** map spreads credit, reduce gathers it (week 1).
- **Confusion matrix:** rows are the actual class, columns the predicted class (week 5).

Problem 1 [37 pts]: Frequent Pattern Mining & Word Count

> We have 4 transactions as follows:
> T0 = {A, B, C, D}
> T1 = {A, C, E, D}
> T2 = {A, E, D}
> T3 = {A, B, C, E, D}
> The minimum support is 70% and the minimum confidence is 80%.

Setup. 4 transactions × 70% = 2.8, so an itemset needs a count of **at least 3** (3/4 = 75% passes; 2/4 = 50% fails). Counting single items is just Word Count over the transactions: map emits ⟨item, 1⟩ and reduce sums.

Item	A	B	C	D	E
Count	4	2	3	4	3
Support	100%	50% ✗	75%	100%	75%

(a) [4] Write frequent 1-itemsets.

> **Source:** W3 p.42 support, and frequent = support ≥ minsup · W4 p.37–40 + W4 p.43 frequent 1-itemsets = "word count with combine" (that's the *Word Count* in the title)
> **Notes:** Week 3 › Frequent patterns · Week 4 › Phase 1: word count

Answer: Frequent 1-itemsets = { {A}, {C}, {D}, {E} }

B is dropped because 2/4 = 50% < 70%.

(b) [8] Write 2-itemsets of the self-join result. Also write the candidate 2-itemsets.

> **Source:** W3 p.65 Step 1 self-join, Step 2 pruning · W3 p.69 C2 = all pairs of L1
> **Notes:** Week 3 › Candidate generation (self-join + prune)

Answer:

- 2-itemsets of self-join result = { {A,C}, {A,D}, {A,E}, {C,D}, {C,E}, {D,E} }
- Candidate 2-itemsets = { {A,C}, {A,D}, {A,E}, {C,D}, {C,E}, {D,E} }

How:

- Self-joining L1 pairs every frequent item with every later one: 4 items give C(4,2) = 6 pairs.
- Pruning removes any itemset that has an infrequent subset. Every 1-subset here (A, C, D, E) is frequent, so nothing is pruned and the candidates equal the self-join result.

(c) [4] Write frequent 2-itemsets.

> **Source:** W3 p.70–72 scan the DB, count, keep ≥ min support → L2
> **Notes:** Week 3 › Full worked example

Candidate	Transactions containing it	Count	Support
{A,C}	T0, T1, T3	3	75% ✓
{A,D}	T0, T1, T2, T3	4	100% ✓
{A,E}	T1, T2, T3	3	75% ✓
{C,D}	T0, T1, T3	3	75% ✓
{C,E}	T1, T3	2	50% ✗
{D,E}	T1, T2, T3	3	75% ✓

Answer: Frequent 2-itemsets = { {A,C}, {A,D}, {A,E}, {C,D}, {D,E} }

(d) [9] Write 3-itemsets of the self-join result. Also write the candidate 3-itemsets and frequent 3-itemsets.

> **Source:** W3 p.65 abc+abd → abcd; acde pruned because ade ∉ L3 (same logic as {A,C,E} pruned because {C,E} ∉ L2) · W3 p.73–74 C3 → L3 · W4 p.27–35 second full example
> **Notes:** Week 3 › Candidate generation · Week 4 › Another Example

Answer:

- 3-itemsets of self-join result = { {A,C,D}, {A,C,E}, {A,D,E} }
- Candidate 3-itemsets = { {A,C,D}, {A,D,E} }
- Frequent 3-itemsets = { {A,C,D}, {A,D,E} }

How:

1. **Self-join L2.** Join two 2-itemsets only when their first item matches. This is the slides' rule: abc+abd gives abcd, but abc+ace is not joined.
   - {A,C}+{A,D} → {A,C,D}
   - {A,C}+{A,E} → {A,C,E}
   - {A,D}+{A,E} → {A,D,E}

    - {C,D} and {D,E} have no partner with the same first item.
2. **Prune.** Each 3-itemset must have all of its 2-subsets in L2.
    - {A,C,D}: AC ✓, AD ✓, CD ✓ → keep.
    - {A,C,E}: **CE ✗** (CE has only 50% support) → pruned.
    - {A,D,E}: AD ✓, AE ✓, DE ✓ → keep.
3. **Scan the DB.**
    - {A,C,D} is in T0, T1, T3: count 3, 75% ✓.
    - {A,D,E} is in T1, T2, T3: count 3, 75% ✓.
4. **Next level.** Self-joining L3 = {ACD, ADE} gives nothing (the prefixes AC and AD differ), so the algorithm stops. The largest frequent itemsets are 3-itemsets.

(e) [12] Write all the rules with support and confidence.

Source: W3 p.43 support = P(X∪Y), confidence = P(Y|X), e.g. Beer → Diaper (60%, 100%) · W4 p.5 same slide again
Notes: Week 3 › Association rules. The slides show only the Beer/Diaper example; trying every subset as a rule is the textbook method, explained there

> Rule1: { } -> { }, xx%, yy% ... where xx% is support and yy% is confidence.

Method:

- For each frequent itemset l with $|l| \geq 2$, try every rule $X \rightarrow (l - X)$, where X is a non-empty proper subset of l.
- support = count(l) / 4.
- confidence = count(l) / count(X). Keep the rule only if confidence ≥ 80%.
- Counts used: A=4, C=3, D=4, E=3, AC=3, AD=4, AE=3, CD=3, DE=3, ACD=3, ADE=3.

Answer: 12 strong rules

#	Rule	Support	Confidence
1	{C} → {A}	75%	3/3 = 100%
2	{A} → {D}	100%	4/4 = 100%
3	{D} → {A}	100%	4/4 = 100%
4	{E} → {A}	75%	3/3 = 100%
5	{C} → {D}	75%	3/3 = 100%
6	{E} → {D}	75%	3/3 = 100%
7	{C} → {A, D}	75%	3/3 = 100%
8	{A, C} → {D}	75%	3/3 = 100%
9	{C, D} → {A}	75%	3/3 = 100%
10	{E} → {A, D}	75%	3/3 = 100%
11	{A, E} → {D}	75%	3/3 = 100%
12	{D, E} → {A}	75%	3/3 = 100%

Rejected rules (confidence 75% < 80%; listed so you can see that every rule was checked):

From	Rejected rules
{A,C}, {A,E}, {C,D}, {D,E}	A→C, A→E, D→C, D→E
{A,C,D}	A→CD, D→AC, AD→C
{A,D,E}	A→DE, D→AE, AD→E

In every rejected rule the left side is A or D, which appear in all 4 transactions (or AD, which has count 4), so confidence = 3/4.

Problem 2 [25 pts]: Inverted Index & PageRank

> We have 4 documents:
> D0: BTS BLACK PINK MUSIC AWARD
> D1: BLACK PINK COLOR
> D2: SONG BLACK PINK
> D3: MUSIC BTS
> D0 has hyperlinks to D2. D1 has a hyperlink to D0, D3. D2 has a hyperlink to D0, D1. D3 has a hyperlink to D2.

(a) [4] Let 4 Mappers M0–M3 take documents D0–D3 respectively. Write the output (list of ⟨key, value⟩ pairs) of each mapper to come up with the Inverted Index.

> **Source:** W1 p.21 Inverted Index: map emits (word, filename)
> **Notes:** Week 1 › Inverted Index

Answer: each mapper emits ⟨word, document⟩ for every word in its document.

- M0 = ⟨BTS, D0⟩, ⟨BLACK, D0⟩, ⟨PINK, D0⟩, ⟨MUSIC, D0⟩, ⟨AWARD, D0⟩
- M1 = ⟨BLACK, D1⟩, ⟨PINK, D1⟩, ⟨COLOR, D1⟩
- M2 = ⟨SONG, D2⟩, ⟨BLACK, D2⟩, ⟨PINK, D2⟩
- M3 = ⟨MUSIC, D3⟩, ⟨BTS, D3⟩

(b) [4] We have 2 Reducers R0 and R1. R0 takes ⟨key, (value1, value2, …)⟩ pairs if the first character of the key is from 'A' to 'J'. R1 takes the rest. Write the output of each reducer.

> **Source:** W1 p.21 reduce outputs (word, sorted filenames) · W2 p.27–28 the partitioner decides which reducer gets a key (here the A–J rule replaces the default hash)
> **Notes:** Week 1 › Inverted Index · Week 2 › Partition & shuffle

Answer:

- R0 = ⟨AWARD, (D0)⟩, ⟨BLACK, (D0, D1, D2)⟩, ⟨BTS, (D0, D3)⟩, ⟨COLOR, (D1)⟩
- R1 = ⟨MUSIC, (D0, D3)⟩, ⟨PINK, (D0, D1, D2)⟩, ⟨SONG, (D2)⟩

How:

- The shuffle groups all values with the same key.
- A, B and C fall in A–J, so AWARD, BLACK, BTS and COLOR go to R0. M, P and S go to R1.

- Each reducer outputs its keys in sorted order, with a sorted document list for each word.

(c) [4] PageRank. Write the output of each mapper.

> **Source:** W1 p.22 the formula, I(p), c_j · W1 p.23 Map = "distribute PageRank credit to link targets" (W2 p.16 repeats it)
> **Notes:** Week 1 › PageRank

> $r_i = d \cdot [\Sigma_{p_j \in I(p_i)} r_j / c_j] + (1 - d) \cdot 1/|D|$
> d = 0.8, and d is multiplied only by reducers. Each document's initial PageRank is 10. Mappers M0–M3 take D0–D3. Round to the nearest tenth.

Answer: each mapper splits its page's rank evenly over that page's out-links and emits ⟨target, r_j / c_j⟩. There is no d at this step, because only reducers apply d.

Mapper	Page (rank, out-links c_j)	Output
M0	D0 (10, 1 link)	⟨D2, 10⟩
M1	D1 (10, 2 links)	⟨D0, 5⟩, ⟨D3, 5⟩
M2	D2 (10, 2 links)	⟨D0, 5⟩, ⟨D1, 5⟩
M3	D3 (10, 1 link)	⟨D2, 10⟩

In a real implementation each mapper also passes its adjacency list along (for example ⟨D0, [D2]⟩) so the next iteration still knows the links. The question only asks for the credit pairs.

(d) [4] We have 4 Reducers R0–R3. Write the output of each reducer. Ignore the second part of the equation.

> **Source:** W1 p.23 Reduce = "gather up PageRank credit from multiple sources to compute new PageRank value"
> **Notes:** Week 1 › PageRank

Answer: reducer R_i receives all credit sent to D_i, sums it, and multiplies by d = 0.8.

Reducer	Input	Computation	Output
R0	⟨D0, (5, 5)⟩	0.8 × 10	⟨D0, 8.0⟩
R1	⟨D1, (5)⟩	0.8 × 5	⟨D1, 4.0⟩
R2	⟨D2, (10, 10)⟩	0.8 × 20	⟨D2, 16.0⟩
R3	⟨D3, (5)⟩	0.8 × 5	⟨D3, 4.0⟩

(e) [8] One more cycle of Map/Reduce with the same mappers and reducers. Write the output of the mappers and reducers.

> **Source:** W1 p.23 "iterate until convergence": feed the (d) output back into the mappers
> **Notes:** Week 1 › PageRank

Answer: use the ranks from (d) as the new inputs.

Mapper	Page (rank, out-links)	Output
M0	D0 (8.0, 1)	⟨D2, 8.0⟩
M1	D1 (4.0, 2)	⟨D0, 2.0⟩, ⟨D3, 2.0⟩
M2	D2 (16.0, 2)	⟨D0, 8.0⟩, ⟨D1, 8.0⟩
M3	D3 (4.0, 1)	⟨D2, 4.0⟩

Reducer	Input	Computation	Output
R0	⟨D0, (2.0, 8.0)⟩	0.8 × 10	⟨D0, 8.0⟩
R1	⟨D1, (8.0)⟩	0.8 × 8	⟨D1, 6.4⟩
R2	⟨D2, (8.0, 4.0)⟩	0.8 × 12	⟨D2, 9.6⟩
R3	⟨D3, (2.0)⟩	0.8 × 2	⟨D3, 1.6⟩

No rounding was needed; every value is already exact to one decimal.

(f) [1] Write the sequence of documents when a user types BLACK PINK and clicks "search".

> **Source:** W1 p.21 index lookup · W1 p.22 PageRank = chance a random surfer lands on the page, i.e. its importance
> **Notes:** Week 1 › Inverted Index · Week 1 › PageRank

Answer: D2 → D0 → D1

How:

1. **Inverted index lookup.** BLACK appears in (D0, D1, D2) and PINK in (D0, D1, D2). The intersection is D0, D1, D2; D3 contains neither word.
2. **Rank by the latest PageRank from (e).** D2 = 9.6 > D0 = 8.0 > D1 = 6.4.

Problem 3 [7 pts]: The Recommendation System

> "Inha Cinema" has 6 customers C1–C6. C1 watched M1, M4, M6. C2 watched M2, M3, M5, M6. C3 watched M2, M4, M5, M6. C4 watched M1, M4, M5, M6. C5 watched M4, M5, M6. C6 watched M3, M4, M5, M6. Inha Cinema builds a recommendation list for C5 by finding the 3 customers most similar to C5, then the movies those 3 watched that C5 has not. Similarity = Jaccard coefficient.

Jaccard(X, Y) = |X ∩ Y| / |X ∪ Y|. C5 = {M4, M5, M6}.

(a) [5] Calculate the Jaccard coefficients between C5 and C1–C6.

> **Source:** W2 p.11 / W2 p.13 Jaccard = |Ci ∩ Cj| / |Ci ∪ Cj| on customer vectors
> **Notes:** Week 2 › Collaborative Filtering with Jaccard

Pair	Intersection	Union	Jaccard
(C5, C1)	{M4, M6} → 2	{M1, M4, M5, M6} → 4	**2/4 = 0.5**
(C5, C2)	{M5, M6} → 2	{M2, M3, M4, M5, M6} → 5	**2/5 = 0.4**
(C5, C3)	{M4, M5, M6} → 3	{M2, M4, M5, M6} → 4	**3/4 = 0.75**
(C5, C4)	{M4, M5, M6} → 3	{M1, M4, M5, M6} → 4	**3/4 = 0.75**
(C5, C6)	{M4, M5, M6} → 3	{M3, M4, M5, M6} → 4	**3/4 = 0.75**

(b) [2] List the movies on the recommendation list for C5.

> **Source:** W2 p.10 / W2 p.12 movie recommendation by collaborative filtering (recommend what similar customers watched)
> **Notes:** Week 2 › Building the recommendation list. The slides give the formula and vectors only; the top-k procedure is the textbook one

Answer: M1, M2, M3

How:

- The 3 most similar customers are C3, C4 and C6 (0.75 each; C1 = 0.5 and C2 = 0.4 are lower, so there is no tie at the cut-off).
- Movies they watched that C5 has not:
  - C3 → M2
  - C4 → M1
  - C6 → M3
- The union is {M1, M2, M3}.

Problem 4 [15 pts]: Decision Tree

> The table below is a training set D of class-labeled tuples. The class label buys_computer has two values (yes, no). Attributes: age, income, student, credit_rating.

RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no

> $\log_2$ n table:

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$\log_2$ n	0	1	1.6	2	2.3	2.6	2.8	3	3.2	3.3	3.5	3.6	3.7	3.8	3.9	4	4.1	4.2

> $\text{Info}(D) = -\Sigma\, p_i \log_2(p_i)$
> $\text{Info_A}(D) = \Sigma\, (|D_j|/|D|) \times \text{Info}(D_j)$
> $\text{Gain}(A) = \text{Info}(D) - \text{Info_A}(D)$

Trick for the table: $-(a/n)\cdot\log_2(a/n) = (a/n)\cdot(\log_2 n - \log_2 a)$. That way you only ever look up logs of whole numbers.

(a) [2] What is Info(D)? Show all your work.

> **Source:** W5 p.27 Info(D) formula · W5 p.29–30 Info(D) = 0.940 for the full 14-row table (the HW table is its first 8 rows, see W5 p.12)
> **Notes:** Week 5 › Worked example

D has 4 yes (RIDs 3, 4, 5, 7) and 4 no (RIDs 1, 2, 6, 8).

$\text{Info}(D) = -(4/8)\cdot\log_2(4/8) - (4/8)\cdot\log_2(4/8) = (4/8)(\log_2 8 - \log_2 4) + (4/8)(\log_2 8 - \log_2 4) = 0.5(3-2) + 0.5(3-2) =$ **1.0**

(b) [8] What are Info_age(D) and Info_income(D)? Show all your work.

> **Source:** W5 p.31–34 Info_age(D) built term by term (= 0.694 on 14 rows) · W5 p.28 the $\log_2$ table
> **Notes:** Week 5 › Worked example · Week 5 › $\log_2$ table trick

Partition by age:

age	RIDs	yes / no	Info(D_j)
youth	1, 2, 8	0 / 3	0 (pure)
middle_aged	3, 7	2 / 0	0 (pure)
senior	4, 5, 6	2 / 1	see below

$\text{Info}(2 \text{ yes}, 1 \text{ no}) = (2/3)(\log_2 3 - \log_2 2) + (1/3)(\log_2 3 - \log_2 1) = (2/3)(1.6-1) + (1/3)(1.6-0) = 0.4 + 0.533 =$ **0.933**

$\text{Info_age}(D) = (3/8)(0) + (2/8)(0) + (3/8)(0.933) =$ **0.35**

Partition by income:

income	RIDs	yes / no	Info(D_j)
high	1, 2, 3	1 / 2	0.933 (same shape as 2/1)
medium	4, 8	1 / 1	(1/2)(1−0) + (1/2)(1−0) = 1.0
low	5, 6, 7	2 / 1	0.933

$\text{Info_income}(D) = (3/8)(0.933) + (2/8)(1.0) + (3/8)(0.933) = 0.35 + 0.25 + 0.35 =$ **0.95**

With exact logs instead of the table: Info(2,1) = 0.918, Info_age = 0.344, Info_income = 0.939.

(c) [4] What are Gain_age(D) and Gain_income(D)? Show all your work.

> **Source:** W5 p.35 Gain(age) = 0.940 − 0.694 = 0.246 · W5 p.36 gains of the other attributes
> **Notes:** Week 5 › Information gain

- Gain(age) = Info(D) − Info_age(D) = 1.0 − 0.35 = **0.65** (exact: 0.656)
- Gain(income) = Info(D) − Info_income(D) = 1.0 − 0.95 = **0.05** (exact: 0.061)

(d) [1] Which attribute can be the splitting criterion between age and income?

> **Source:** W5 p.27 "select the attribute with the highest information gain" · W5 p.37 the tree splits on age
> **Notes:** Week 5 › Information gain

Answer: age, because it has the higher information gain (0.65 vs 0.05).

Splitting on age makes youth and middle_aged pure; only senior still needs another split.

Problem 5 [16 pts]: Confusion Matrix

> **Source:** W5 p.40 confusion matrix (rows actual, columns predicted; TP/FN/FP/TN) · W5 p.41 accuracy, class-imbalance problem (fraud) · W5 p.42 precision, recall
> **Notes:** Week 5 › Confusion matrix · Week 5 › Evaluation metrics

Layout as in the slides: **rows = actual**, **columns = predicted**.

- TP = actual yes and predicted yes
- FN = actual yes but predicted no
- FP = actual no but predicted yes
- TN = actual no and predicted no

Formulas:

- Accuracy = (TP + TN) / All
- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)

(a) [8] Model M1 detects fraudulent credit card transactions. The test data has 1000 transactions, 5 of them actually fraudulent. M1 flagged 10 as fraudulent, 3 of which are real fraud. Complete the confusion matrix and compute accuracy, precision and recall.

Filling the cells:

- TP = 3 (flagged and real)
- FP = 10 − 3 = 7 (flagged but not fraud)
- FN = 5 − 3 = 2 (real fraud that was missed)
- TN = 1000 − 3 − 7 − 2 = 988

Actual \ Predicted	Fraud = Yes	Fraud = No	Total
Fraud = Yes	3	2	5
Fraud = No	7	988	995
Total	10	990	1000

- **Accuracy** = (3 + 988) / 1000 = **99.1%**
- **Precision** = 3 / 10 = **30%**
- **Recall** = 3 / 5 = **60%**

This is the class-imbalance case: accuracy looks excellent, but only 30% of the alerts are real fraud and 40% of the frauds are missed.

(b) [8] Model M2 decides loan approvals. The test data has 1000 loan reviews, 502 of them actual approvals. M2 predicted 500 approvals, 499 of which are real approvals. Complete the confusion matrix and compute accuracy, precision and recall.

Filling the cells:

- TP = 499
- FP = 500 − 499 = 1
- FN = 502 − 499 = 3
- TN = 1000 − 499 − 1 − 3 = 497

Actual \ Predicted	Loan = Yes	Loan = No	Total
Loan = Yes	499	3	502
Loan = No	1	497	498
Total	500	500	1000

- **Accuracy** = (499 + 497) / 1000 = **99.6%**
- **Precision** = 499 / 500 = **99.8%**
- **Recall** = 499 / 502 = **99.4%** (99.40%)

Here the classes are balanced, so a high accuracy really does mean a good model; compare with (a).

Answer summary (to copy onto the paper)

Q	Answer
1a	{A}, {C}, {D}, {E}
1b	Self-join = Candidates = {A,C}, {A,D}, {A,E}, {C,D}, {C,E}, {D,E}
1c	{A,C}, {A,D}, {A,E}, {C,D}, {D,E}
1d	Self-join: {A,C,D}, {A,C,E}, {A,D,E} · Candidates: {A,C,D}, {A,D,E} · Frequent: {A,C,D}, {A,D,E}
1e	C→A (75%,100%) · A→D (100%,100%) · D→A (100%,100%) · E→A (75%,100%) · C→D (75%,100%) · E→D (75%,100%) · C→AD · AC→D · CD→A · E→AD · AE→D · DE→A (all 75%,100%)
2a	M0: ⟨BTS,D0⟩⟨BLACK,D0⟩⟨PINK,D0⟩⟨MUSIC,D0⟩⟨AWARD,D0⟩ · M1: ⟨BLACK,D1⟩⟨PINK,D1⟩⟨COLOR,D1⟩ · M2: ⟨SONG,D2⟩⟨BLACK,D2⟩⟨PINK,D2⟩ · M3: ⟨MUSIC,D3⟩⟨BTS,D3⟩
2b	R0: ⟨AWARD,(D0)⟩⟨BLACK,(D0,D1,D2)⟩⟨BTS,(D0,D3)⟩⟨COLOR,(D1)⟩ · R1: ⟨MUSIC,(D0,D3)⟩⟨PINK,(D0,D1,D2)⟩⟨SONG,(D2)⟩
2c	M0 ⟨D2,10⟩ · M1 ⟨D0,5⟩⟨D3,5⟩ · M2 ⟨D0,5⟩⟨D1,5⟩ · M3 ⟨D2,10⟩
2d	R0 ⟨D0,8.0⟩ · R1 ⟨D1,4.0⟩ · R2 ⟨D2,16.0⟩ · R3 ⟨D3,4.0⟩
2e	M0 ⟨D2,8.0⟩ · M1 ⟨D0,2.0⟩⟨D3,2.0⟩ · M2 ⟨D0,8.0⟩⟨D1,8.0⟩ · M3 ⟨D2,4.0⟩ · R0 ⟨D0,8.0⟩ · R1 ⟨D1,6.4⟩ · R2 ⟨D2,9.6⟩ · R3 ⟨D3,1.6⟩
2f	D2, D0, D1
3a	C1 0.5 · C2 0.4 · C3 0.75 · C4 0.75 · C6 0.75
3b	M1, M2, M3
4a	Info(D) = 1.0
4b	Info_age = 0.35 · Info_income = 0.95
4c	Gain(age) = 0.65 · Gain(income) = 0.05
4d	age
5a	TP 3, FN 2, FP 7, TN 988 · Acc 99.1% · Prec 30% · Rec 60%
5b	TP 499, FN 3, FP 1, TN 497 · Acc 99.6% · Prec 99.8% · Rec 99.4%